WORKSHOP CONECT INOVE

Do zero à loja

Seu primeiro app com React Native e Expo

Workshop Conect Inove | 5 encontros | Google Meet

Encontro 1	Encontro 2	Encontro 3	Encontro 4	Encontro 5
26/10 19:30	**27/10** 19:30	**29/10** 19:30	**05/11** 19:30	**07/11** 13:00



Quem vai ensinar

Otávio Rafael

- Portfólio: portfolio.otaviorafael.com.br
- Blog: blog.otaviorafael.com.br



Portfólio



Blog

Roteiro do workshop

ENCONTRO	DATA	TEMA
1	Seg 26/10, 19:30	Ambiente e primeira tela
2	Ter 27/10, 19:30	Estado e lista de tarefas
3	Qui 29/10, 19:30	Salvar dados e preparar para as lojas
4	Qui 05/11, 19:30	Build e Google Play
5	Sáb 07/11, 13:00	Apple App Store (demonstrativo) e próximos passos

O que vamos construir

Conect Tarefas

- Adicionar tarefas
- Marcar como concluída
- Apagar
- Filtrar (todas, pendentes, feitas)
- **Salvar no celular**
- E depois: publicar nas lojas

O caminho de um app até a loja

```
Código  →  Testar  →  Build  →  Loja  →  Revisão  →  Usuários
VS Code    Expo Go     EAS      Play/Apple
```

Encontros 1 a 3: código e teste. Encontros 4 e 5: build e lojas.

React Native, Expo, Expo Go, EAS

React Native:
JavaScript + React para criar apps Android e iOS

Expo:
ferramentas que simplificam tudo (sem Android Studio e sem Mac)

Expo Go:
app do celular para ver seu projeto na hora

EAS:
serviço na nuvem que gera o app final e envia às lojas

Encontro 1

Ambiente e primeira tela

SEG 26/10 · 19:30

Checklist de instalação

- Node.js (LTS), VS Code e Git (`node -v`, `git --version`)
- Contas gratuitas: GitHub e Expo
- Expo Go no celular
- Celular e PC no **mesmo Wi-Fi**

Node.js VS Code Git GitHub Expo

Criando o projeto

terminal

```
npx create-expo-app@latest conect-tarefas --template blank
cd conect-tarefas
npx expo start
```

Escaneie o QR Code com o **Expo Go** (Android) ou com a **Câmera** (iPhone).

Não abriu? `npx expo start --tunnel` ou aperte `w` para abrir no navegador.

Arquivos do projeto

ARQUIVO	FUNÇÃO
`App.js`	Tela principal
`app.json`	Nome, ícone, identificador, versão
`package.json`	Bibliotecas do projeto
`assets/`	Imagens

Componentes do React Native

COMPONENTE	PARECIDO COM	USO
`View`	div	Caixa
`Text`	p	Todo texto
`TextInput`	input	Digitar
`TouchableOpacity`	button	Tocar
`FlatList`	lista	Muitos itens

Estilos e Flexbox

App.js

```
const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#F4F6FB' },
  titulo: { fontSize: 30, fontWeight: 'bold' },
  linha: { flexDirection: 'row' },
});
```

- camelCase: `backgroundColor`
- Números sem "px"
- Layout em **coluna** por padrão

Etapa 1: a tela

App.js

```
export default function App() {
  return (
    <View style={styles.container}>
      <Text style={styles.titulo}>Conect Tarefas</Text>
      <View style={styles.linhaNova}>
        <TextInput style={styles.input} placeholder="Nova tarefa..." />
        <TouchableOpacity style={styles.botao}>
          <Text style={styles.botaoTexto}>+</Text>
        </TouchableOpacity>
      </View>
    </View>
  );
}
```

Git e GitHub

Git:
histórico do código, como um "salvar jogo"

GitHub:
site que guarda os repositórios na internet

Commit:
um ponto de salvamento

Branch:
uma linha paralela de trabalho

Pegando o projeto do workshop

github.com/otaviorafaeldev/workshop-conect-inove-react-native



Repositório do workshop

Opção 1 (sem Git): botão verde **Code** > **Download ZIP**

Opção 2: clonar com Git (próximo slide)

Clonando e rodando

terminal

```
git clone https://github.com/otaviorafaeldev/workshop-conect-inove-react-native.git
cd workshop-conect-inove-react-native/conect-tarefas
npm install
npx expo start
```

Sua branch para os desafios

terminal

```
git config --global user.name "Seu Nome"
git config --global user.email "seu@email.com"

git switch -c desafios/maria-silva

git add .
git commit -m "Desafio 1: troquei as cores"
```

- Nome da branch: **seu nome**, minúsculas, sem acentos
- Nunca resolva na `main`
- Backup online: faça um **fork** e use `git push`

ENCONTRO 1 SEG 26/10 · 19:30 PARA CASA

Desafio 1 (para casa)



- Troque as cores do app
- Troque o título pelo seu nome
- Adicione mais um card
- Mande o print no grupo
- Salve em um **commit** na sua branch

Encontro 2

Estado e lista

TER 27/10 · 19:30

2

O que é estado?

A **memória** do componente. Mudou o estado, a tela se redesenha.

App.js

```
const [texto, setTexto] = useState('');
const [tarefas, setTarefas] = useState([]);
```

- `texto`: valor atual
- `setTexto`: função que muda o valor
- `''`: valor inicial

Adicionar tarefa

App.js

```
function adicionar() {
  if (texto.trim() === '') return;
  const nova = { id: Date.now().toString(), titulo: texto.trim(), feita: false };
  setTarefas([nova, ...tarefas]);
  setTexto('');
}
```

Regra de ouro: nunca mexa direto na lista. Crie uma nova.

Concluir e apagar

App.js

```
function alternar(id) {
  setTarefas(tarefas.map((t) ⇒ (t.id ⩵ id ? { ...t, feita: !t.feita } : t)));
}

function apagar(id) {
  setTarefas(tarefas.filter((t) ⇒ t.id ≠ id));
}
```

- `map`: transforma cada item
- `filter`: mantém só alguns itens

FlatList

App.js

```
<FlatList
  data={tarefas}
  keyExtractor={(item) => item.id}
  renderItem={({ item }) => (
    <Text>{item.titulo}</Text>
  )}
/>
```

- `data`: os dados
- `keyExtractor`: id único de cada item
- `renderItem`: o que desenhar



Desafio 2 (para casa)

- Mostre o total de tarefas embaixo do título
- Bônus: mude a cor do card quando a tarefa estiver feita

Encontro 3

Salvar dados e identidade do app

QUI 29/10 · 19:30

3

O problema

Fechou o app? **As tarefas sumiram.**

O estado vive só na memória. Precisamos guardar no celular.

terminal

```
npx expo install @react-native-async-storage/async-storage
```

`expo install` escolhe a versão compatível com o seu projeto.

useEffect + AsyncStorage

App.js

```
// Ao abrir o app: ler
useEffect(() => {
  async function carregar() {
    const salvo = await AsyncStorage.getItem(CHAVE);
    if (salvo) setTarefas(JSON.parse(salvo));
    setCarregou(true);
  }
  carregar();
}, []);

// Quando a lista mudar: salvar
useEffect(() => {
  if (carregou) AsyncStorage.setItem(CHAVE, JSON.stringify(tarefas));
}, [tarefas, carregou]);
```

Contador e filtros

App.js

```
const visiveis = tarefas.filter((t) => {
  if (filtro === 'pendentes') return !t.feita;
  if (filtro === 'feitas') return t.feita;
  return true;
});
const qtdFeitas = tarefas.filter((t) => t.feita).length;
```

Identidade do app (app.json)

app.json

```
"name": "Conect Tarefas",
"version": "1.0.0",
"ios":     { "bundleIdentifier": "com.SEUNOME.tarefas", "buildNumber": "1" },
"android": { "package": "com.SEUNOME.tarefas", "versionCode": 1 }
```

O identificador é único no mundo e não muda depois de publicar.

Formato: minúsculas, sem espaços e sem acentos.

Ícone e verificação

- PNG **1024 x 1024**, quadrado
- Substituir `assets/icon.png`

terminal

```
npx expo-doctor
```

Objetivo: "No issues detected".

ENCONTRO 3 QUI 29/10 · 19:30 PARA CASA

Desafio 3 (para casa)



- Crie o seu ícone
- Defina o seu identificador `com.seunome.tarefas`
- Rode o `expo-doctor`
- Crie sua conta em expo.dev

Encontro 4

Build e Google Play

QUI 05/11 · 19:30



Por que precisamos de build?

SISTEMA	ARQUIVO	USO
Android	`.apk`	Instalar direto para testar
Android	`.aab`	**Formato exigido pela Play Store**
iOS	`.ipa`	App Store e TestFlight

O EAS gera tudo na nuvem. Sem Android Studio. Sem Mac.

ENCONTRO 4 QUI 05/11 · 19:30

Preparando o EAS

terminal

```
npm install -g eas-cli
eas login
eas init
```

app.json

```
"build": {
  "preview":    { "distribution": "internal", "android": { "buildType": "apk" } },
  "production": { "autoIncrement": true }
}
```

Gerando os builds

terminal

```
# APK para testar no celular
eas build --platform android --profile preview

# AAB para a Play Store
eas build --platform android --profile production
```

- Primeira vez: aceite **gerar a keystore** (a chave que assina o app)
- Plano gratuito: fila e limite mensal. Teste bem antes de gerar build!

Conta Google Play Console

- play.google.com/console
- **US$ 25**, pagamento único
- Titular **maior de 18 anos**
- Verificação de identidade (pode levar dias)
- Conta pessoal nova: **teste fechado com 12 testadores por 14 dias** antes da produção

Passo a passo na Play Console

1. Criar app
2. Painel de tarefas: privacidade, anúncios, classificação, público-alvo, segurança dos dados
3. Ficha da loja: nome, descrições, ícone, prints
4. Subir o `.aab` numa trilha de teste
5. Enviar para análise

O **primeiro** envio é manual, pelo site.

Ficha da loja (Google)

ITEM	REGRA
Nome	até 30 caracteres
Descrição curta	até 80
Descrição completa	até 4.000
Ícone	512 x 512
Imagem de destaque	1024 x 500
Prints	mínimo 2

Trilhas de teste

Interno:
até 100 testadores, disponível em minutos

Fechado:
testadores convidados (regra dos 12 testadores)

Aberto:
qualquer pessoa pode entrar

Produção:
todo mundo

Política de privacidade

Obrigatória nas duas lojas, **mesmo que o app não colete nada**.

> O aplicativo Conect Tarefas não coleta, não compartilha e não envia dados pessoais. As tarefas ficam apenas no dispositivo do usuário.

Hospede de graça: Notion público, Google Sites ou GitHub Pages.

Encontro 5

Apple App Store (demonstrativo) e próximos passos

SÁB 07/11 · 13:00

Google vs Apple

	GOOGLE PLAY	APPLE
Custo	US$ 25 uma vez	**US$ 99 por ano**
Revisão	Horas a dias	1 a 2 dias
Teste	Trilhas internas	TestFlight
Rigor	Médio	Alto
Precisa de Mac?	Não	Não (com EAS)

Passo a passo na Apple

Aqui só mostramos o caminho. Quem quiser, segue o tutorial depois.

1. Apple ID com verificação em duas etapas
2. Inscrição no Apple Developer Program
3. App Store Connect: criar o app (nome, Bundle ID, SKU)
4. `eas build --platform ios --profile production`
5. `eas submit --platform ios`
6. TestFlight: testar no iPhone
7. Preencher a página do app e enviar para revisão

Build e envio para a Apple

terminal

```
eas build --platform ios --profile production
eas submit --platform ios
```

- O EAS cria **certificados e perfis** automaticamente
- O build aparece no **TestFlight** em alguns minutos
- Credenciais: `eas credentials`

Página do app na App Store

- Prints do iPhone (6,9")
- Descrição e palavras-chave (100 caracteres)
- URL de suporte
- URL da política de privacidade
- Privacidade do app e classificação etária
- Notas para a revisão

Por que apps são rejeitados?

MOTIVO	COMO EVITAR
App simples demais (regra 4.2)	Dê utilidade e capricho
Trava ou fecha	Teste em vários aparelhos
Sem política de privacidade	URL válida
Prints falsos	Prints reais do app
Informação incompleta	Preencha tudo

Rejeitou? Leia o motivo, corrija e reenvie.

Atualizando o app

1. Mude a `version` no `app.json`
2. Novo build de produção
3. Enviar para as lojas e passar pela revisão

Bônus: EAS Update

terminal

```
eas update
```

Mudanças só de JavaScript chegam aos usuários sem nova revisão.

Checklist final

- [ ] App testado (Expo Go + APK / TestFlight)
- [ ] Nome, ícone e identificador
- [ ] Contas: Expo, Google Play, Apple
- [ ] Prints, descrições, política de privacidade
- [ ] Builds de produção (`.aab` e `.ipa`)
- [ ] Questionários respondidos
- [ ] Enviado para revisão

ENCONTRO 5 SÁB 07/11 · 13:00

Próximos passos

- Navegação entre telas (Expo Router)
- Editar tarefa, datas, categorias, notificações
- Modo escuro
- Login e nuvem (Firebase ou Supabase)
- Monetização
- **Criar e publicar o seu próprio app**

Artigo: publicar na Play Store e App Store

Docs do Expo

React Native

Portfólio

CONECT INOVE

Obrigado!

Agora é com você: publique o seu app

Conect Inove

